

Inference Staking Protocol

A General Purpose On-Chain Primitive for Off-Chain Coordination

Authors: Sean Smith, Sam Hogan, Jerry Ong, Tommy Johnson, Taylor Johnson

Date: November 1, 2025

Abstract

We present an open-source protocol implementation leveraging crypto-economic principles to coordinate a decentralized network of participants while ensuring honesty. The core protocol is flexible and can be extended to support a variety of different applications and use cases. In our case, this protocol powers a distributed network of GPU devices that deliver AI inference on open-source large language models (LLMs). The protocol employs a “proof-of-stake” system to ensure participant integrity and provides a delegator staking mechanism to introduce unique crypto-economic incentives. A token reward system is supported to drive growth and adoption and the protocol supports a novel stablecoin-based revenue sharing mechanism. Multiple safeguards and security mechanisms exist to ensure protocol integrity, mitigate risk, and guard against bad actors. We have integrated this protocol with our live distributed inference system, proving this can work at scale for real world applications. In this paper, we present the design rationale and technical implementation details of the Inference Staking Protocol and describe how we have integrated it into our distributed AI inference system.

1. Introduction

In early 2024, [Inference.net](https://inference.net) was founded based on the observation that at any given moment there exists a large amount of idle GPU compute in the world. Despite the extreme demand for new high-end GPUs for model training from the leading AI labs, in practice as much as half of all GPU compute sits idle, unused, at any given moment [1, 2, 3]. While model pre-training is compute intensive and requires cutting edge, co-located datacenter-grade GPUs to continue pushing the frontiers of model capability, the requirements for AI inference are very different.

AI inference workloads are short-lived, often stateless, and can run on lower-tier GPU devices. Consumer and prosumer-grade GPUs, such as NVIDIA’s GeForce RTX 40-series, are widespread in gaming PCs and personal workstations. High-end graphics cards like the RTX 4090 or 4080 deliver training-level performance and are often comparable to lower-tier datacenter GPUs when idle at home. NVIDIA ships tens of millions of these GPUs per year [4].

Apple, meanwhile, has added tens of millions of M-series chips per year since introducing their custom silicon [5, 6]. Conservatively, there are probably in excess of 100 million lower-end GPUs globally, between gaming PCs and M-series Apple hardware, all of which are capable of serving inference on small models at competitive speeds (~40 tokens/second and faster).

Another important trend of the last few years is the rise of open-source AI models. Meta's Llama series of models which first launched in 2023 [7] was closely followed by releases from Mistral AI [8]. In the last 2 years, Alibaba's Qwen series of models [9], DeepSeek [10], and more recently NVIDIA's Nemotron model family [11] and OpenAI's own OSS model releases [12] have continued to push the frontier of open model performance forward. Many of these open models score very competitively against top closed source models on leaderboard evaluations [13]. Some of the open models are relatively small, i.e. less than ~30B parameters, and may be quantized or distilled into even smaller models which can run optimally on low-end GPU devices.

Finally, as model capabilities continue to improve and AI workflows become more integrated throughout the global economy, AI inference demand will rise exponentially. Market forecasts predict the AI inference market to grow near ~20% CAGR through 2030, with the overall market size exceeding ~\$250B [14, 15]. Much of this inference will be dominated by cutting edge frontier models, but much of it will spill over into increasingly capable open-source models which can often be very competitive in terms of cost, quality, or latency.

Together, these observations compelled us to build a protocol to aggregate globally distributed GPU compute for the task of serving open-source LLM inference.

2. Background

When we began, the idea of coordinating decentralized compute resources was not new and there were many existing protocols we drew inspiration from. The most prominent examples are Bitcoin and Ethereum which use distributed computing resources to maintain decentralized ledgers. Many additional projects have built on those foundations to provide general compute, file storage, 3D rendering, and most recently, various AI-centered capabilities such as GPU rentals and model training. In particular, Akash Network [16], Bittensor [17], IO Network [18], Gensyn [19], Nous Research [20] and Prime Intellect [21] are all actively exploring the space of direct GPU rentals and/or distributed model training. Each of these projects have developed bespoke solutions to distributed coordination, worker verification, and incentives/rewards. However, none of them are exclusively focused on serving LLM inference. In addition, there are many other decentralized physical infrastructure projects (DePIN), such as Helium [22], Hivemapper [23], and Grass [24], which are also active in the design space of coordinating off-chain activities with on-chain verification and payment. We saw an opportunity for a more general coordination protocol which we could adapt to our specific use case of serving distributed LLM inference.

In recent years, proof-of-stake has matured as a proven mechanism for secure blockchain consensus, with leading blockchains from Tendermint-based systems to Ethereum to Solana

adopting it as their primary consensus mechanism [25, 26, 27]. Proof-of-stake requires participants to stake capital in order to participate in the system, and transparent, provable verification mechanisms hold them accountable—any violations of protocol rules risk permanent slashing of staked capital. Moreover, next generation blockchains such as Solana are battle-tested and offer secure programming environments for shipping smart contracts at scale with low transaction costs and fast finality.

We set out to design and implement a novel proof-of-stake program on Solana which was general purpose and could be adapted to our specific use case of serving LLM inference across a distributed network of devices. The protocol would coordinate rewards and payments between participants, support staking for security and growth incentives, and provide flexible controls for diverse applications.

3. Implementation

Our protocol is a general-purpose staking primitive built on the Solana network, designed to coordinate operator-delegator relationships and distribute on-chain rewards for verified off-chain workloads. The architecture provides a secure, flexible, and efficient framework for proof-of-stake coordination, token-based reward distribution, stablecoin (USDC) revenue sharing, and security safeguards.

The system is composed of four primary on-chain accounts:

- **PoolOverview**: Global, singleton account storing protocol state and parameters.
- **OperatorPool**: Account representing operators who complete off-chain work tasks.
- **StakingRecord**: Account representing a staked position in an **OperatorPool**.
- **RewardRecord**: Account representing an epoch reward payment to operators.

These accounts work together to manage global protocol state, operator pools, user stake positions, and epoch reward distributions.

3.1 Protocol Management

The **PoolOverview** account is a singleton account which is created once by the protocol admin after the program is first deployed. This account stores important configuration parameters and protocol state. Several security safeguards and administrative controls are provided by this account, which allow the protocol admin to manage settings such as staking, withdrawals and operator pool registration.

In order to support more fine-grained access controls throughout the protocol lifecycle, several specific authority lists are defined for specific protocol functions. These are halting, slashing and reward distribution. The program admin may specify and update specific public keys that are authorized to perform these actions.

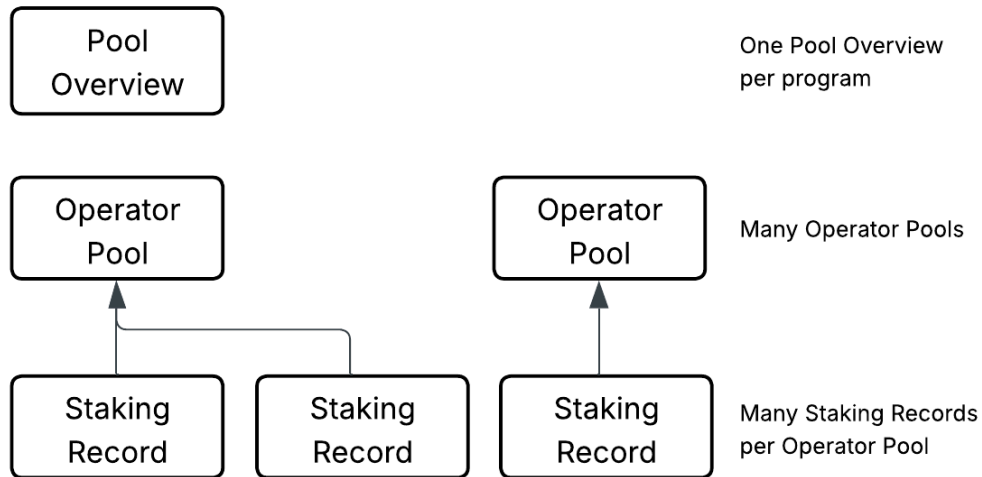


Figure 1: Overview of primary program accounts.

3.2 Protocol Token

The protocol supports a native token which is specified when the `PoolOverview` account is created. The protocol can be configured in one of two modes when deployed which allow it to use a native SPL token or USDC as the native protocol token. This is controlled by two parameters which must be specified when creating the pool overview: `is_token_mint_usdc` and `token_rewards_enabled`. These settings are immutable and cannot be changed after they are set. These parameters allow the protocol, generally, to operate in one of two modes:

- **Native Token:** In this mode, the protocol supports a native token and token rewards, in addition to independent stablecoin revenue sharing. Delegators may stake with operators and earn a share of token rewards and USDC revenue. Token rewards are distributed according to the token reward emissions schedule, which is encoded on-chain. The native token could be configured as wrapped SOL (wSOL).
- **USDC Only:** In this mode, the protocol operates without a native token and functions primarily to distribute USDC revenue to the group of operators. Additional features like operator staking and slashing may also be enabled, but are not required. If staking is enabled, staking is conducted in USDC.

3.3 Operator Lifecycle

The primary participant in the protocol is the operator, who performs off-chain work and is compensated via on-chain rewards and revenue payments. The operator lifecycle proceeds as follows:

- **Registration:** An operator registers an operator pool in the protocol. On registration the operator must submit a payment of `operator_pool_registration_fee`, which is

payable in the protocol native token (may be USDC if the protocol is configured in USDC mode). This fee amount is specified on the [PoolOverview](#) account.

- **Operator Staking:** Operators may be required to stake their own pool. The [PoolOverview](#) provides a parameter `min_operator_token_stake` which specifies this amount—if specified and operators fail to stake this threshold additional delegations to their pool will be prevented and they may be excluded from work assignments.
- **Operator Pool Management:** The operator can manage various settings on their pool, such as reward and revenue commission rates, whether fees are auto-staked to their staking account, and whether delegation is allowed to their pool or not. Operators may also manage and update specific wallets such as the admin wallet and staking account associated with their pool.
- **Operation:** The operator pool is active and the operator is eligible to complete any off-chain tasks assigned to them. The program admin is responsible for delegating these tasks and paying operators for their work. Operators will receive token rewards and stablecoin revenue payments in proportion to their work, at the discretion of the protocol admin.
- **Delegator Staking:** If the protocol is configured to allow delegation, and the operator enables delegation on their pool, external token holders may stake tokens to a given operator. This increases the operator pool's total stake, which is important for applications that use stake to determine work assignments. In addition, delegators will receive a share of token rewards and stablecoin payments the pool receives, proportional to their stake in the pool and the pool's commission rate settings.
- **Halting and Slashing:** In the rare event that the protocol admin identifies an operator is in violation of protocol rules they may be halted and/or slashed. In a halting event, the operator can no longer perform any staking actions or withdraw any earnings from the pool. External delegators are unaffected. Halting effectively freezes a pool to allow for further investigation. If wrongdoing is confirmed, an arbitrary amount of the operator's stake may be slashed, which confiscates this stake and transfers it to an external wallet configured by the program admin. An operator pool may later be unhalted, and return to normal function.
- **Claiming Earnings:** Operators may claim earnings paid out to their pool at any time, and the protocol supports various options to allow this. For token rewards, operators can specify if the rewards should auto-compound their staked position in the pool or accumulate in a separate account. For stablecoin payments, they will always accrue in a separate account. Operators may withdraw accrued earnings at any time directly to an external wallet. Importantly, withdrawing USDC earnings may occur independently of any changes in their staked position in the pool.
- **Pool Closure:** Operators may close their pools and cease operation at any time. Program admins must observe pool closures and avoid dispatching work assignments to closed pools. Once closed, an operator may fully withdraw their stake and any accrued earnings from the pool. Delegators, likewise, are advised to unstake from the pool. Once all stake has been withdrawn from a closed pool, the operator has the option of closing their staking record account and also sweeping any remaining USDC dust from the pool.

3.4 Delegator Lifecycle

If the protocol is configured to allow staking, external token holders may stake tokens to operator pools. This feature is important when program admins want to allow protocol operation with token incentives—i.e. in cases where the protocol should assign more work or more rewards to operators with more stake. Delegators may stake and unstake from any pools as long as the pools allow external delegation. Unstaking will require waiting for staked tokens to cooldown based on the delay configured on the [PoolOverview](#) account.

Any earnings paid to the pool will auto-compound to all pool delegators in proportion to their pool stake and the pool's commission rate settings. Any earned USDC revenue share may be withdrawn at any time, via the [ClaimUsdcEarnings](#) instruction, without impacting the delegator's staked position. Once a delegator has fully unstaked from a pool, they may optionally close their staking record account to reclaim the account rent.

3.5 Staking Mechanics

The protocol abstracts staking into a share-based system, similar to a liquid staking pool. When an operator or delegator stakes tokens to a pool, the tokens are converted to pool shares according to the following formula:

$$\text{shares_minted} = \frac{\text{token_amount} \times \text{total_shares}}{\text{total_staked_amount}}$$

Likewise, when tokens are unstaked, shares are converted back to tokens according to the following formula:

$$\text{tokens_unstaked} = \frac{\text{shares_amount} \times \text{total_staked_amount}}{\text{total_shares}}$$

This design allows for token rewards to efficiently compound the staked positions of all delegators. When rewards are paid out, operator pool stake increases but no shares are minted.

The protocol admin can manage two delay periods for unstaking, delays for operators and delegators. These delay periods are intended as security measures and help to prevent pool stake from rotating too frequently.

3.6 Stablecoin Revenue Sharing Mechanics

Stablecoin revenue is paid into the protocol by admins to operators, in proportion to off-chain work tasks completed by the operators. Operators may choose to share stablecoin revenue with delegators (to attract more stake) or to retain it all for themselves. If an operator enables stablecoin revenue sharing, then stablecoin revenue is shared with all delegators in the pool proportional to their stake in the pool. This is tracked by an independent accounting mechanism, to allow stablecoin withdrawals to occur independently of staking actions.

Specifically, each `OperatorPool` maintains a `cumulative_usdc_per_share` index, and each `StakingRecord` maintains a `last_settled_usdc_per_share` index. The staking record index is effectively a checkpoint of when USDC earnings were last settled to that staker, in reference to the earned USDC per share for the pool. The key implementation detail is that whenever a staking event happens (i.e. any action that mutates staking shares), any outstanding USDC earnings must first be settled to that staking record.

The full flow is as follows. First, some amount of USDC revenue is distributed to an `OperatorPool`. The change in USDC per share for that pool is calculated as:

$$\Delta_{\text{index}} = \frac{\text{delegator_usdc_amount} \times 10^{18}}{\text{total_shares}}$$

$$\text{cumulative_usdc_per_share} += \Delta_{\text{index}}$$

The 10^{18} denominator is a scaling factor to maintain precision for fractional values of USDC.

This index update is made every time USDC revenue is paid to an operator pool. Many USDC payments can be made to a pool, which will continue to increment this USDC per share index. Separately, a delegator may choose to join the pool and stake tokens. For this delegator, their settlement checkpoint will be initialized to the current pool USDC per share. Whenever there is a delta between the pool USDC per share and the checkpoint on a `StakingRecord`, the delegator has unclaimed USDC earnings. For this new delegator, there is no delta and therefore no unclaimed earnings yet.

Another delegator, however, may have staked in the past and therefore has USDC earnings. Their unclaimed earnings are given by:

$$\text{pending_earnings} = \frac{(\text{pool.index} - \text{user.last_settled_index}) \times \text{user.shares}}{10^{18}}$$

Importantly, USDC earnings are decoupled from staking actions. To support this, the protocol settles any outstanding USDC earnings before any staking action occurs, but does not require outstanding earnings to be withdrawn. This allows delegators to stake, earn rewards and USDC, claim USDC, and then later unstake, if they wish. Token rewards can only be withdrawn by unstaking. To accomplish this, the `StakingRecord` tracks any settled but unclaimed USDC earnings via an `accrued_usdc_earnings` field. Whenever a staking event occurs, this field is incremented by any pending earnings and the `StakingRecord` USDC per share index is reset back to the `OperatorPool` `cumulative_usdc_per_share` index.

For protocols that issue a native token, the USDC revenue sharing feature effectively turns the native protocol token into a USDC yielding asset.

3.7 Reward Epochs

Rewards and stablecoin revenue are distributed to the protocol by the protocol admin in “epochs”. An epoch is an arbitrary time interval, which is determined off-chain by the protocol administrator. The lifecycle for earning distributions is as follows:

1. An epoch is marked as finalizing by the protocol admin.
2. An epoch **RewardRecord** account is created by the admin, which records the total token reward and USDC distribution for that epoch. This account stores merkle root(s) which encode the full distribution to all operator pools in that epoch. The creation of this account on-chain effectively finalizes the epoch.
3. Operators must each claim their rewards and revenue for an epoch by submitting a merkle proof as proof of their inclusion in the epoch payment. Submitting this claim instruction effectively transfers their pool-share earnings from the global on-chain vaults to vaults specific to their pool.

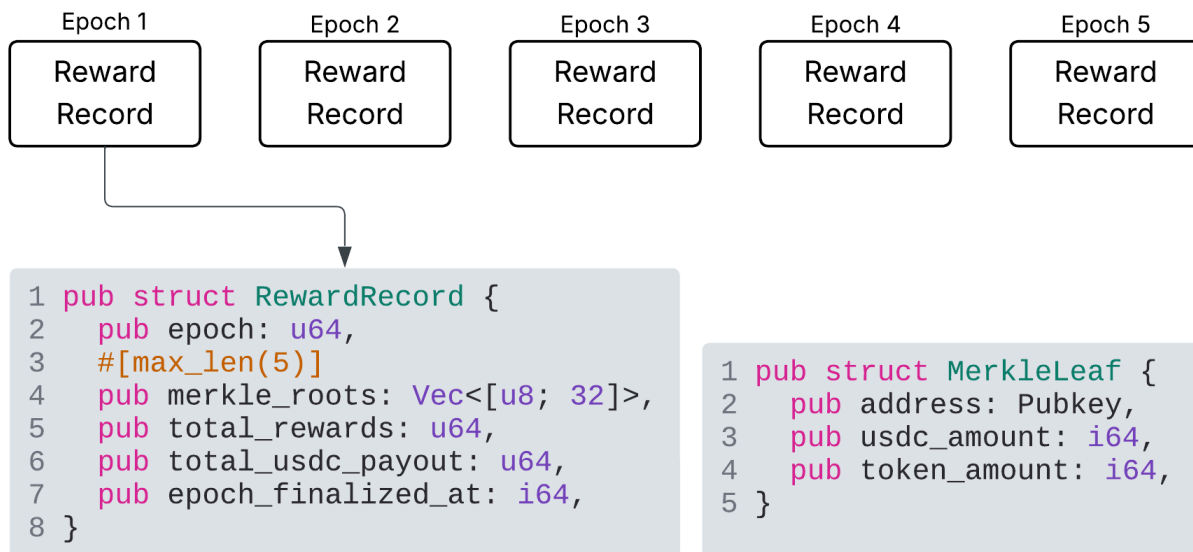


Figure 2: Overview of **RewardRecord** account data structure and merkle tree compression.

Submitting a reward claim transaction (the **AccrueReward** instruction) for an operator pool triggers several accounting settlements:

1. Token rewards for the operator and delegator are calculated based on the pool’s token reward commission rate. Token rewards are auto-staked for the operator if they have auto-staking enabled or otherwise they are transferred directly to the operator’s token commissions vault. Delegator rewards are transferred to a dedicated vault.

2. USDC revenue earnings for the operator and delegators are calculated based on the pool's USDC commission rate. Any operator USDC earnings are transferred directly to their earnings vault. Delegator USDC is deposited in a separate on-chain vault.
3. The pool's cumulative USDC per share index is updated.
4. Global accounting state updates occur on the [PoolOverview](#) and [OperatorPool](#).
5. Any pending commission rate changes for the pool take effect.

The reward claim transactions are permissionless and may be invoked by any user. Protocol administrators may choose to “crank” these reward claim transactions on behalf of pool operators as a convenience.

All tokens are stored in on-chain PDA vaults that are associated either with the global [PoolOverview](#) or individual [OperatorPool](#) accounts, establishing a transparent and auditable record of operator earnings.

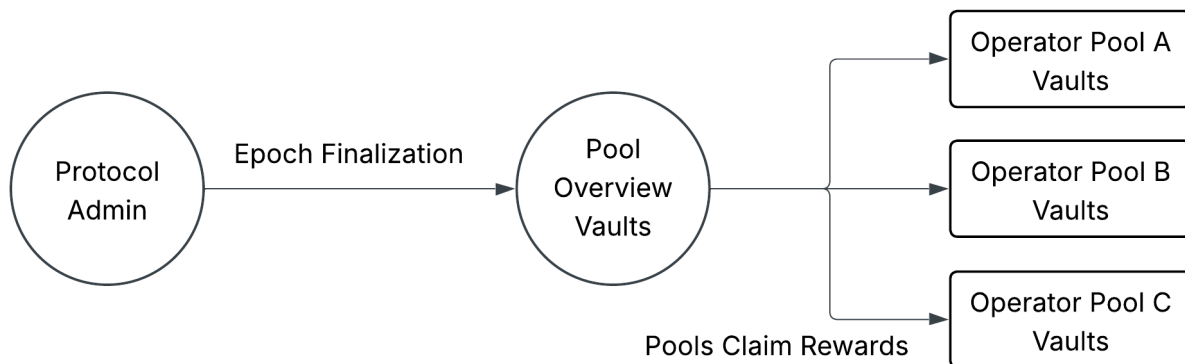


Figure 3: Overview of protocol fund flows. All vaults are Solana PDAs.

3.8 Token Reward Emissions

The protocol supports a token emissions schedule, encoded on-chain. This is useful when a project issues a native token and wants to create early growth incentives for adoption. For example, a protocol may choose to allocate 10% of total token supply as reward emissions, and distribute 2% per year for the first 5 years. These values can be encoded on-chain and the expected per-epoch value will be validated when each epoch is finalized. Validating the reward emissions on-chain increases transparency and accountability for projects that choose to issue token rewards. In this flow, protocol admins are responsible for depositing sufficient token rewards into the program reward vault, otherwise the [CreateRewardRecord](#) instruction will fail.

Protocol admins may choose to deploy a program with zero rewards or with the token reward function completely disabled, if they wish.

3.9 Protocol Security

The protocol implements several security measures to ensure safe operation. First, the [PoolOverview](#) account provides several failsafes that may be activated by the protocol admin to immediately halt staking, withdrawals, and reward accrual operations. These may be engaged in any emergency situation. Second, the protocol supports halting and slashing operations against operator pools, which ensure the honesty of these potentially anonymous actors. Importantly, protocol admins must implement measures to verify the integrity of protocol participants. In the event a dishonest participant is identified, that operator may be halted and/or slashed.

- **Halting:** Halting an operator effectively freezes their pool and prevents the operator from withdrawing any earnings or un-staking tokens, while the situation is investigated. A halted operator may be unhalted at any time. Delegators are unaffected by halting and may withdraw their stake from a halted operator pool.
- **Slashing:** Slashing confiscates operator stake from an operator pool and transfers it to a specified withdrawal wallet. The amount of tokens slashed is up to the discretion of the protocol admin, up to a maximum of the operator's stake.

The risk of capital loss via slashing is the primary deterrent against protocol violations. This principle has been battle tested by leading proof-of-stake networks now for many years and is a robust security measure for systems which must tolerate a diverse set of potentially anonymous participants.

3.10 Off-Chain Work Assignments

The on-chain protocol implementation is agnostic to the nature of the workloads the operators are responsible for completing. This decoupling allows the protocol admins maximum flexibility in terms of how they organize and assign tasks to operators. In our case, for instance, we route inference requests to operators based on their pool stake and hardware capacity, and assign different models to workers based on the current supply/demand profile of the network and the worker's GPU profile. Payments for inference tasks depend on the details of the inference request: the current pricing for that model and the input/output tokens for that request.

4. Decentralization

The current protocol implementation does require a single protocol admin to retain control of many protocol functions. This may be a regulatory concern and also is a single point of failure. It's worth a brief exploration of how these responsibilities could be decentralized in the future. Taking inspiration from similar work such as Jito's Node Consensus Network [28], Wormhole [29], Pyth Network [30], and others, one might implement an off-chain coordination layer external to the program where multiple independent nodes would be responsible for observing and voting on off-chain state. Consensus could be coordinated by simple on-chain voting mechanisms, where a configurable quorum of nodes would be required to trigger actions such

as reward payouts, halting and slashing events. While out of the scope of this initial protocol implementation paper, such a design would provide a reasonable path toward long-term decentralization.

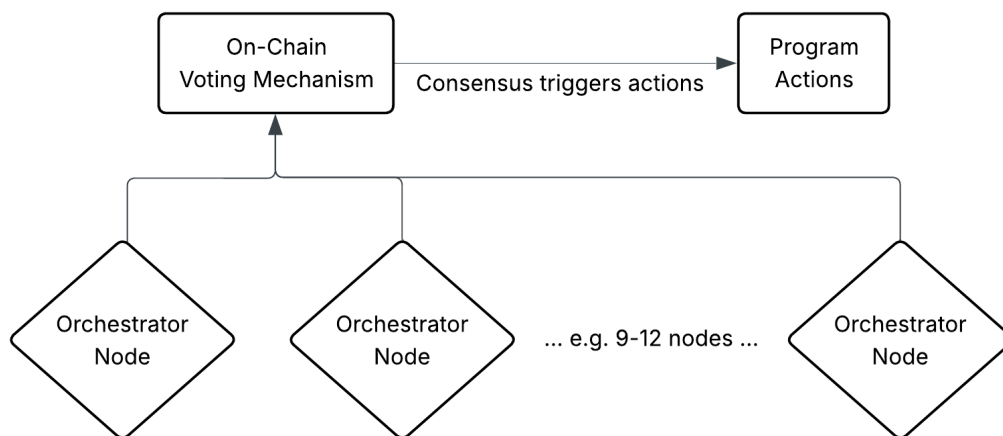


Figure 4: General architecture for a decentralized version of the protocol.

5. Applications

At [Inference.net](https://inference.net) we have deployed the Inference Staking Protocol to power our decentralized AI inference system. We have deployed this program to Solana Devnet and operated the system in a production-like environment for over 6 months now. In this environment, over 3,500 operators have joined, over 8,000 delegators have staked tokens, ~1,500 reward epochs have been completed, and a total of almost 4,000,000 on-chain transactions have been finalized. Various state and health checks for the system have been continuously monitored and remain healthy. No accounting or protocol violations have been observed.

Our integration of the staking protocol introduces novel mechanics in the off-chain dynamics, which demonstrate the overall flexibility and usefulness of the system. Three such mechanics worth highlighting are stake-weighted routing, inference verification, and reputation scores.

5.1 Stake-Weighted Routing

Our distributed inference network has many thousands of devices connected at any moment. In order to align incentives between GPU operators and network integrity, we introduced a stake-weighted inference routing system. This routing engine is managed within our infrastructure and routes requests preferentially to operators with the most stake first. As highly staked operators reach capacity and become saturated, requests spill over to lower staked operators. Operators are paid directly for processing requests, creating an incentive for operators to acquire more stake to receive more requests. Moreover, highly staked operators have more capital at risk which drives a stronger disincentive against breaking protocol rules. Overall, these tokenomics create mutually aligned incentives among token holders: Operators

want to accumulate more stake to receive more inference tasks, more stake increases their incentive to behave honestly, and delegators want to select the highest quality operators to delegate their tokens to.

5.2 Inference Verification

We have implemented a novel inference verification mechanism called LOGIC which we employ to identify dishonest operators who may be modifying our inference client or experimenting with other techniques to return fraudulent responses to inference requests. LOGIC applies statistical analysis to LLM response log probability distributions to reliably verify if a given response has been produced authentically [31]. For our use case, this is the primary violation we need to guard against. As a first measure, operators must build up a minimum trust score in order to be routed traffic. Secondly, any verification failures may trigger halting and slashing actions against the offending operator.

5.3 Reputation Scores

In addition to inference verification, we've also implemented a reputation scoring system which provides additional information to potential delegators and gives us richer information to feed into our routing engine and reward distributions. In particular, we track reliability and uptime scores for all operators. Reliability scores give us a secondary factor after inference verification to consider when routing requests and uptime scores allow us to reward operators who maintain idle capacity GPUs on the network, which helps us deal with unexpected demand spikes. Specifically, we can introduce uptime reward incentives for operators running specific models which gives us more control in managing network compute supply and maintaining a stable supply baseline when inference demand is low.

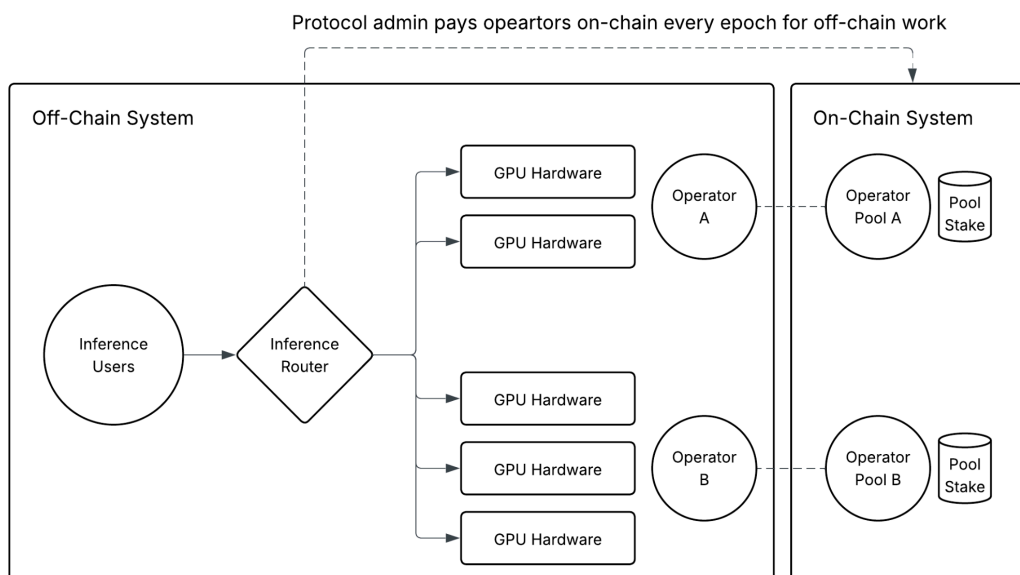


Figure 5: [Inference.net](#) protocol implementation, highlighting on-chain/off-chain separation.

6. Summary

Here, we have presented a general-purpose proof-of-stake protocol which can be used to coordinate off-chain activities using on-chain crypto-economic principles and described how we have integrated this protocol to power our decentralized AI inference network.

Our deployment of the Inference Staking Protocol demonstrates it can operate efficiently and safely at scale. We have conducted one full audit of the program code and may conduct additional audits in the future. The full source code, including client tests and an SDK, has been open-sourced on GitHub at <https://github.com/context-labs/inference-staking> and is available for other teams to use. You can see the current protocol in operation, and create an operator pool or participate in staking, at <https://stake.devnet.inference.net>.

References

1. GPU-as-a-Service, Harvesting Idle Capacity, and the Rise of Alternative Processing Units. Ng S.T. Chong, February 6, 2025
(<https://c3.unu.edu/blog/gpu-as-a-service-harvesting-idle-capacity-and-the-rise-of-alternative-processing-units/>)
2. Big Cloud deploys thousands of GPUs for AI – yet most appear under-utilized. Tobias Mann, January 15, 2024
(https://www.theregister.com/2024/01/15/cloud_providers_gpu_analysis/)
3. Idle GPUs Highlight Compute Allocation Inefficiency: Impact on Crypto Mining and AI Trading. May 23, 2025
(<https://blockchain.news/flashnews/idle-gpus-highlight-compute-allocation-inefficiency-impact-on-crypto-mining-and-ai-trading>)
4. NVIDIA Claws Away More PC GPU Market Share From Intel & AMD As Shipments Reach 74.7 Million Units In Q2 2025. Hassan Mujtaba, August 30, 2025
(<https://wccftech.com/nvidia-claws-away-gpu-market-share-intel-amd-shipments-reach-74-7-million-q2-2025>)
5. Gartner Says Worldwide PC Shipments Increased 4.8% in First Quarter of 2025. April 14, 2025
(<https://www.gartner.com/en/newsroom/press-releases/2025-04-14-gartner-says-worldwide-pc-shipments-increased-4-point-8-percent-in-first-quarter-of-2025>)
6. Macs just keep flying off store shelves. Ed Hardy, January 12, 2022
(https://www.cultofmac.com/news/apple_mac_shipments_2021_2020)
7. Introducing LLaMA: A foundational, 65-billion-parameter large language model. February 24, 2023 (<https://ai.meta.com/blog/large-language-model-llama-meta-ai>)
8. Mistral AI makes its first large language model free for everyone. Devin Coldewey, September 27, 2023
(<https://techcrunch.com/2023/09/27/mistral-ai-makes-its-first-large-language-model-free-for-everyone>)

9. Tongyi Qianwen (Qwen) (<https://www.alibabacloud.com/en/solutions/generative-ai/qwen>)
10. Deepseek AI (<https://huggingface.co/deepseek-ai>)
11. NVIDIA Nemotron: Open and efficient multimodal models for agentic AI.
(<https://www.nvidia.com/en-us/ai-data-science/foundation-models/nemotron>)
12. Introducing gpt-oss: gpt-oss-120b and gpt-oss-20b push the frontier of open-weight reasoning models. August 5, 2025 (<https://openai.com/index/introducing-gpt-oss>)
13. LLM Leaderboard - Comparison of over 100 AI models from OpenAI, Google, DeepSeek & others (<https://artificialanalysis.ai/leaderboards/models>).
14. AI Inference Market to Reach USD 349.49 Billion by 2032 Driven by Growing Need for Real-Time Processing and Low-Latency AI Applications | Research by SNS Insider. SNS Insider pvt ltd, September 24, 2025
(<https://finance.yahoo.com/news/ai-inference-market-reach-usd-071900459.html>)
15. AI Inference Market (And Segment Forecasts 2025 - 2030) Size, Share & Trends.
(<https://www.grandviewresearch.com/industry-analysis/artificial-intelligence-ai-inference-market-report>)
16. Akash Network: Decentralized Cloud Infrastructure Marketplace. Overclock Labs, May 13, 2018 (<https://ipfs.io/ipfs/QmVwsi5kTrg7UcUEGi5UfdheVLBWoHjze2pHy4tLqYvLYv>)
17. Bittensor: A Peer-to-Peer Intelligence Market. Yumao Rao
(<https://bittensor.com/whitepaper>)
18. Understanding io.net: A Comprehensive Overview. Whynonah, May 2, 2025
(<https://messari.io/report/understanding-io-net-a-comprehensive-overview>)
19. The Gensyn Protocol (<https://docs.gensyn.ai/the-gensyn-protocol>)
20. Democratizing AI: The Psyche Network Architecture. Nous Research Team, May 14, 2025 (<https://nousresearch.com/nous-psyche/>)
21. State-of-the-art in Decentralized Training. Johannes, Sami, Vincent, April 23, 2024
(<https://www.primeintellect.ai/blog/our-approach-to-decentralized-training>)
22. Helium: A Decentralized Wireless Network. Amir Haleem Andrew Allen Andrew Thompson Marc Nijdam Rahul Garg, November 14, 2018 (<http://whitepaper.helium.com>)
23. Hivemapper: A global, decentralized mapping network for everyone.
(<https://docs.hivemapper.com/>)
24. Grass (<https://grass-foundation.gitbook.io/grass-docs>)
25. Tendermint: Consensus without Mining. Jae Kwon, 2014
(<https://tendermint.com/static/docs/tendermint.pdf>)
26. Ethereum Proof-of-stake (PoS). October 21, 2015
(<https://ethereum.org/developers/docs/consensus-mechanisms/pos>)
27. Solana: A new architecture for a high performance blockchain v0.8.14. Anatoly Yakovenko
(<https://github.com/solana-labs/whitepaper/blob/master/solana-whitepaper-en.pdf>)
28. NCNs: An overview of the Node Consensus Network (NCN) component.
(<https://www.jito.network/docs/restaking/core-concepts/ncn>)
29. Introduction to Wormhole. (<https://wormhole.com/docs/protocol/introduction>)
30. Pyth Network: A First-Party Financial Oracle. Pyth Data Association version 2.0
September 28, 2023 (<https://pythdataassociation.com/whitepaper.pdf>)
31. Inference.net LOGIC: Log Probability Verifier (<https://github.com/context-labs/logic>)